HARDWARE SECURITY LAB REPORT

Side-Channel Analysis and Fault Injection Experiments

Abian Morina

Assan Jallow

19 of March 2026

Contents

1 Introduction

This report documents a series of hardware security laboratories focused on how practical attacks can exploit weaknesses in real embedded implementations. Rather than treating cryptography as a purely algorithmic problem, these labs examine the physical realities of embedded devices, how operations on internal states can manifest as measurable side effects in power consumption and timing, and how deliberately induced timing violations can corrupt program execution.

The experiments were carried out using the **ChipWhisperer** platform, which provides the measurement and control infrastructure required for both classes of attacks covered in this report. ChipWhisperer supports capturing analog power traces from an embedded target, enabling statistical evaluation of leakage. It also supports generating precisely timed **clock glitches** that can move the target briefly outside its reliable operating window, producing faults that can invalidate security-relevant checks.

The labs are organized into two main tracks:

- **SCA101 (Side-Channel Analysis)**: Power-based attacks ranging from simple timing/control-flow leakage to full AES key recovery via **Differential Power Analysis (DPA)** and **Correlation Power Analysis (CPA)**.
- **Fault101 (Fault Injection)**: Clock glitch attacks that induce transient malfunctions in microcontroller execution, including bypass of authentication logic.

Across the entire set of labs, the work targets embedded firmware running on microcontrollers (typically XMEGA) connected to ChipWhisperer capture hardware. For each lab, the report explains the objective and target behavior, the attack methodology and leakage model, the tools and setup used in the workflow, the results obtained, and the limitations and interpretation necessary to draw correct conclusions from real measurements.

2 Lab 2.1B: Power Analysis for Password Bypass

2.1 Topic Introduction

This lab introduces **Simple Power Analysis (SPA)** as a means to recover a secret password from an embedded device that performs a character-by-character password check. The attack exploits the fact that power consumption differs depending on whether each character comparison succeeds or fails, revealing the correct password one character at a time without brute-forcing the entire string.

2.2 Description of the Target

The target runs firmware (`basic-passwdcheck`) that compares user input to a stored 5-character password. The comparison is performed one byte at a time in a loop. When a character matches, the device follows a different code path (or exhibits different power behavior) than when it does not. The device returns a pass/fail result over the SimpleSerial protocol.

2.3 Attack Methodology

1. **Trace acquisition**: Power traces are captured while the target performs password checks. Each trace is associated with a known input string (e.g., a single character or a prefix of the password).
2. **Reference trace**: A reference trace is chosen using an input that is always "wrong" (e.g., a null byte `0x00`).
3. **Difference-of-waveforms**: For each candidate character in the valid set (`a-z`, `0-9`), a trace is captured and the sample-by-sample difference from the reference trace is computed.
4. **Character recovery**: The candidate producing the largest total absolute difference (e.g., **np.sum** (**np.abs**(**diff**))) corresponds to the correct character.
5. **Full password recovery**: The process is repeated position by position, using the already-recovered prefix for each subsequent character.

2.4 Results Obtained

The attack successfully recovered the full 5-character password (e.g., `h0px3`). For each position, one candidate character yielded a distinctly larger difference metric than the others, enabling reliable character-by-character recovery. The difference-of-means trace showed a clear, localized deviation at the time when the device was performing the character comparison.

2.5 Analysis and Interpretation of the Results

The results demonstrate that **control-flow and data-dependent leakage** in power consumption can be exploited to bypass authentication. The attack is non-invasive and requires only power measurements and chosen inputs. The difference-of-waveforms technique amplifies the leakage by subtracting a "wrong" reference trace from a "correct" one, highlighting the time region where the device behavior differs.

2.6 Challenges or Limitations

- Careful handling of the reference trace and prefix is required: when attacking position i, the reference and all candidate inputs must use the same correct prefix for positions $0..i-1$.
- Trace alignment and noise can affect reliability; sufficient traces per character may be needed for robust recovery.

2.7 Key Takeaways

- Simple character-by-character password checks are vulnerable to power side-channel analysis.
- Difference-of-waveforms is an effective technique for identifying data-dependent leakage.
- Constant-time, leakage-resistant implementations are necessary for security-critical firmware.

2.8 Conclusion

This lab demonstrated that a simple password verification routine can be broken using power side-channel analysis. By exploiting data- and control-flow-dependent power differences and applying difference-of-waveforms comparisons, the correct password was recovered one character at a time, confirming the practical risk of non-hardened embedded authentication code.

3 Lab 3.1: Large Hamming Weight Swings

3.1 Topic Introduction

This lab demonstrates **data-dependent power leakage** during cryptographic computation. The goal is to show that a microcontroller's power consumption is measurably influenced by the **Hamming weight** (number of 1-bits) of the data being processed. By inducing a large Hamming-weight contrast in the plaintext (0x00 vs. 0xFF for one byte), the leakage becomes visible in the power trace.

3.2 Description of the Target

The target runs AES (TinyAES128C) on an XMEGA microcontroller. The attacker can choose or observe the plaintext. Power traces are captured during encryption. The experiment focuses on one plaintext byte, which is varied between 0x00 (Hamming weight 0) and 0xFF (Hamming weight 8) to maximize the observable leakage.

3.3 Attack Methodology

1. **Trace acquisition**: Capture many power traces while the target encrypts plaintexts. The chosen byte is randomly 0x00 or 0xFF; the rest of the plaintext can vary.
2. **Grouping**: Partition traces into two sets based on the value of the chosen byte (0x00 vs. 0xFF).
3. **Averaging**: Compute the mean trace for each group, sample by sample, to reduce noise.
4. **Difference-of-means**: Compute $\Delta(t) = \bar{T}_{FF}(t) - \bar{T}_{00}(t)$ and plot it.
5. **Interpretation**: A distinct spike in $\Delta(t)$ at a specific time indicates that power consumption at that moment is correlated with the Hamming weight of the processed byte.

3.4 Results Obtained

The difference-of-means trace showed a **clear, localized spike** early in the waveform, indicating that the device's power consumption is measurably influenced by the data being manipulated. Averaging was essential to suppress random noise and reveal the deterministic data-dependent effect.

3.5 Analysis and Interpretation of the Results

The experiment validates the core assumption of power side-channel analysis: **power consumption correlates with intermediate values** handled by the device. The lab uses a deliberately large Hamming-weight difference to maximize signal visibility; in practice, smaller differences may require more traces or stronger statistical methods.

3.6 Challenges or Limitations

- Measurement interpretation: depending on the shunt resistor sensing arrangement, voltage changes may appear inverted relative to "more power" intuition.
- The analysis targets one byte; extending to multiple bytes or different AES rounds requires additional experimentation.

3.7 Key Takeaways

- Data-dependent leakage is experimentally observable in power traces.
- Averaging and difference-of-means are fundamental techniques for isolating leakage.
- This forms the foundation for more advanced attacks (DPA, CPA).

3.8 Conclusion

This lab confirmed that the target's power consumption exhibits measurable dependence on the Hamming weight of data handled during AES. By partitioning traces and computing a difference-of-means waveform, the data-dependent component emerged clearly, validating Hamming-weight-based leakage assumptions for subsequent DPA/CPA-style attacks.

4 Lab 3.2: Recovering Data from a Single Bit

4.1 Topic Introduction

This lab demonstrates that a **single bit** of an internal AES intermediate (the S-box output) can be used to recover an entire key byte. The attack exploits the fact that, for the correct key guess, the predicted value of that bit aligns with the device's actual behavior; for incorrect guesses, the grouping is effectively random. A Monte Carlo-style analysis shows how the number of encryptions needed to recover the key decreases as the probability of correctly observing that bit increases.

4.2 Description of the Target

The target runs AES-128 (TinyAES128C). Power traces and known plaintexts are available. The attack targets the first-round S-box output for one byte: $v = \text{Sbox}(p \oplus k)$, where p is the known plaintext byte and k is the unknown key byte. One bit of v is used as the leakage model.

4.3 Attack Methodology

1. **AES model**: Implement the intermediate computation $v = \text{Sbox}(p \oplus k)$ using the standard AES S-box.
2. **Bit selection**: Choose one bit of v (e.g., least significant bit) as the leakage hypothesis.
3. **Key recovery**: For each key-byte guess, partition traces by the predicted bit value. The correct guess produces a distinct statistical separation; incorrect guesses produce random-like grouping.
4. **Monte Carlo analysis**: Simulate key recovery with varying "observation accuracy" (probability of correctly observing the bit) and plot encryptions required vs. accuracy.

4.4 Results Obtained

The analysis showed that even with moderate observation accuracy (e.g., above 45%), the number of encryptions required to recover the key drops dramatically compared to very low accuracy (e.g., 20%). At high accuracy (90–100%), only a handful of encryptions are needed. The "Guesses for Single Bit Observation" plot illustrates this inverse relationship.

4.5 Analysis and Interpretation of the Results

The results indicate that **imperfect leakage is still exploitable**: as long as the observed bit is slightly biased toward the truth, repeated measurements and statistics can recover the key. This motivates the transition to power-based DPA, where the "observation" is inferred from power rather than known directly.

4.6 Challenges or Limitations

- The lab assumes the bit can be observed (e.g., from simulation); in practice, power measurements must be used to infer it.
- Ghost peaks and noise can complicate key ranking.

4.7 Key Takeaways

- A single bit of leakage can suffice for key recovery with enough traces.
- The relationship between observation accuracy and required encryptions is highly non-linear.
- This lab bridges to DPA, where power replaces direct bit observation.

4.8 Conclusion

This lab showed that even minimal leakage,modeled as a single bit of an AES internal intermediate,can be leveraged to recover key material. The Monte Carlo analysis reinforced that partial or imperfect leakage remains exploitable, motivating the transition from "known-bit" demonstrations to real power-based differential techniques (DPA).

5 Lab 3.3: DPA on Firmware Implementation of AES

5.1 Topic Introduction

This lab implements a **Differential Power Analysis (DPA)** attack against a real AES-128 firmware implementation. Instead of assuming a known bit value, the attack uses **power measurements** to validate key guesses. Traces are partitioned by a predicted bit of the S-box output; the correct guess produces a clear difference-of-means peak at the time of the S-box operation.

5.2 Description of the Target

The target runs TinyAES128-C on an XMEGA microcontroller. Power traces are captured during encryption with random plaintexts under a fixed secret key. The attack targets the first-round S-box output: $v = \text{Sbox}(p \oplus k)$ for each byte position.

5.3 Attack Methodology

1. **Trace acquisition**: Capture many traces (e.g., 2,500) with known plaintexts and fixed key.
2. **Key-byte attack**: For each key byte (0–15), for each guess $g \in \{0, \ldots, 255\}$:
   - Compute hypothetical intermediate $\hat{v}_i = \text{Sbox}(p_i[b] \oplus g)$ for each trace (where $p_i[b]$ is plaintext byte b in trace i).
   - Partition traces by one bit of $\hat{v}_i$ (e.g., bit 0).
   - Compute mean trace for each partition and form $\Delta(t) = \bar{T}_1(t) - \bar{T}_0(t)$.
   - Score: $\max_t |\Delta(t)|$.
3. **Key recovery**: Select the guess with the largest score for each byte; combine to form the full key.
4. **Ghost peaks**: Some wrong guesses may produce peaks; increasing trace count or using multiple candidates can mitigate this.

5.4 Results Obtained

For each key byte, the correct value (e.g., 0x2B for byte 0) yielded a clearly larger maximum difference than incorrect guesses. The full 128-bit key was recovered and matched the known key. Plots of $\Delta(t)$ for correct vs. incorrect guesses showed a distinct spike for the correct guess.

5.5 Analysis and Interpretation of the Results

The lab demonstrates that **unprotected firmware AES leaks key-dependent information** through power consumption. The DPA attack exploits the correlation between one bit of the S-box output and power at the time of the S-box operation. Ghost peaks can occur but are manageable with sufficient traces or multi-candidate strategies.

5.6 Challenges or Limitations

- Ghost peaks may require more traces or alternative distinguishers.
- Parameter tuning (width, offset for glitch-style attacks) is not applicable here; trace count and alignment are the main variables.

5.7 Key Takeaways

- Classic DPA can recover the full AES key from power traces and known plaintexts.
- The S-box output is a critical leakage point in unprotected implementations.
- Countermeasures (masking, hiding) are necessary for secure embedded crypto.

5.8 Conclusion

This lab implemented classic DPA against a firmware AES implementation and demonstrated full key recovery using differential comparisons of traces. The observed separation for correct key guesses, along with the mention of ghost peaks, highlighted both the effectiveness of DPA and the practical constraints imposed by noise and statistical artifacts, reinforcing the need for side-channel hardened designs.

6 Lab 4.1: Power and Hamming Weight Relationship

6.1 Topic Introduction

This lab refines the leakage model by demonstrating that power consumption at the AES S-box output is **linearly related** to the **Hamming weight** of the intermediate value. Establishing this relationship validates the use of Hamming weight as a leakage model for more powerful attacks such as CPA.

6.2 Description of the Target

The target runs AES-128 (TinyAES128C) on an XMEGA. Power traces are captured during encryption. The S-box output for a selected byte is computed from known plaintext and (for verification) known key. The Hamming weight of that byte (0–8) is the predicted leakage.

6.3 Attack Methodology

1. **Hamming weight computation**: Implement a function to count the number of 1-bits in a byte.
2. **Trace grouping**: Group traces by the Hamming weight of the S-box output for a chosen byte.
3. **Locating the S-box in time**: Subtract the overall mean trace from each group's mean to isolate the S-box-related signal; identify the sample index where the S-box operation occurs.
4. **Power vs. Hamming weight**: Extract the power value at that sample for each Hamming-weight group and plot power vs. Hamming weight (0–8).

6.4 Results Obtained

The plot of power vs. Hamming weight showed an **approximately linear relationship**: higher Hamming weight corresponded to proportionally larger (or smaller, depending on measurement polarity) power. On many ChipWhisperer setups the slope is negative due to shunt-resistor current sensing; on

the ChipWhisperer Nano it can be positive due to an inverting amplifier.

6.5 Analysis and Interpretation of the Results

The linear relationship validates the Hamming-weight leakage model and justifies its use in CPA. Subtracting the overall mean trace is essential to make the small data-dependent component visible amid the larger common-mode structure and noise.

6.6 Challenges or Limitations

- The S-box signal is small relative to the full trace; preprocessing (mean subtraction, averaging) is critical.
- Measurement polarity varies by hardware; interpretation may require sign adjustment.

6.7 Key Takeaways

- Hamming weight is a practical first-order leakage model for embedded crypto.
- Linear power–Hamming-weight relationship supports correlation-based attacks.
- This lab provides the foundation for CPA in Lab 4.2.

6.8 Conclusion

This lab validated that the measured power at the AES S-box processing point correlates strongly with the Hamming weight of the corresponding intermediate. The approximately linear relationship and the discussion of measurement polarity provided justification for using Hamming weight as a predictive leakage model, establishing a solid foundation for correlation-based key recovery in CPA.

7 Lab 4.2: CPA on Firmware Implementation of AES

7.1 Topic Introduction

This lab implements **Correlation Power Analysis (CPA)** to recover the AES-128 key. Instead of partitioning traces by a single bit (DPA), CPA uses the full Hamming-weight model and computes the **Pearson correlation** between predicted leakage and measured power at each time sample. The correct key guess maximizes the correlation at the time of the S-box operation.

7.2 Description of the Target

The target runs TinyAES128-C on an XMEGA. Power traces and known plaintexts are captured. The attack models leakage as the Hamming weight of the first-round S-box output: $\ell(g) = \mathrm{HW}(\mathrm{Sbox}(p \oplus g))$.

7.3 Attack Methodology

1. **Trace acquisition**: Capture traces (e.g., 50 for CPA, fewer than DPA) with known plaintexts.
2. **Correlation computation**: For each key-byte guess g, compute predicted leakage vector $\ell(g)$ for all traces. For each time sample t, let $\mathbf{p}_t$ be the column of measured power values at sample t across traces; compute $\rho_g(t) = \mathrm{corr}(\ell(g), \mathbf{p}_t)$ (Pearson correlation).
3. **Key-byte decision**: The guess with $\max_t |\rho_g(t)|$ is the recovered byte.
4. **Full key**: Repeat for all 16 bytes.

7.4 Results Obtained

CPA produced a clear correlation peak at the S-box operation time. The correct key guess achieved the highest correlation for each byte, enabling full key recovery. CPA typically required fewer traces (e.g.,

∼50) than DPA (e.g., 2,500) due to the more informative Hamming-weight model.

7.5 Analysis and Interpretation of the Results

CPA is more efficient than DPA because it uses the full Hamming-weight information rather than a single bit. The correlation coefficient provides a robust, continuous distinguisher. Ghost peaks can still occur but are often less pronounced than in DPA.

7.6 Challenges or Limitations

- Trace alignment and noise affect correlation magnitude.
- The `cwtraces` package may be required for simulated data; hardware capture avoids this dependency.

7.7 Key Takeaways

- CPA is a powerful, trace-efficient alternative to DPA for AES key recovery.
- The Hamming-weight model and Pearson correlation form a standard attack combination.
- Unprotected firmware AES remains vulnerable to modern SCA techniques.

7.8 Conclusion

This lab demonstrated that correlation power analysis using a Hamming-weight leakage model can recover AES key material effectively. Compared to DPA, CPA was described as more trace-efficient because it uses richer predicted leakage information and quantifies agreement via correlation peaks at the S-box operation time, confirming the practical threat posed by unprotected embedded implementations.

8 Fault 1.1: Introduction to Clock Glitching

8.1 Topic Introduction

This lab introduces **clock glitching** as a fault-injection technique. By briefly violating a microcontroller's clock timing (inserting extra edges or shortening cycles), the device can be induced to skip instructions, corrupt data, or follow incorrect control flow. The lab targets a simple summation loop to demonstrate that glitches can cause incorrect computational results.

8.2 Description of the Target

The target runs `simpleserial-glitch` firmware, which performs a deterministic loop-based calculation and returns the result over SimpleSerial. The computation is designed to be glitchable: a long loop with many instructions provides a large time window for fault injection.

8.3 Attack Methodology

1. **Baseline**: Run the target without glitching to establish the correct result.
2. **Glitch configuration**: Configure ChipWhisperer's glitch module (`scope.glitch`) with parameters such as width, offset, and trigger mode.
3. **Fault injection loop**: For each glitch attempt, send a command to start the computation, insert the glitch, and read the result using `target.simpleserial_read_witherrors()` to handle corrupted responses.
4. **Outcome classification**: Categorize each attempt as normal (correct result), faulted (wrong result), or crash/timeout (target hung).
5. **Parameter sweep**: Systematically vary glitch width and offset to find regions that produce useful faults.

8.4 Results Obtained

Certain glitch parameter combinations produced **faulted outputs** (wrong loop result) while the target remained responsive. Other combinations caused **crashes or timeouts**. The experiment validated that clock glitching can induce controlled computational errors.

8.5 Analysis and Interpretation of the Results

The summation loop is an effective introductory target because it offers a long execution window and any malfunction is easily detected. Real-world attacks often target specific instructions (e.g., a comparison or branch) and require finer timing control.

8.6 Challenges or Limitations

- Glitch parameter space is large; finding "sweet spots" requires systematic search.
- Target resets and error-tolerant communication are essential for automation.

8.7 Key Takeaways

- Clock glitching can cause repeatable computational faults.
- ChipWhisperer's glitch module provides configurable width, offset, and triggering.
- This lab establishes the foundation for more security-relevant fault attacks.

8.8 Conclusion

This lab established clock glitching as a viable fault-injection technique capable of inducing incorrect computation in an embedded target. By configuring glitch parameters and using fault-tolerant communication patterns, the lab showed that useful fault outcomes can be obtained while still enabling automated experimentation and analysis, forming the basis for more targeted fault attacks.

9 Fault 1.2: Clock Glitching to Bypass Password

9.1 Topic Introduction

This lab applies clock glitching to a **password check** in firmware. The goal is to bypass authentication by inducing a fault that causes the target to accept an incorrect password. The attack uses glitch parameters identified in the previous lab and extends the search with an **ext_offset** parameter to improve timing precision.

9.2 Description of the Target

The target runs the same `simpleserial-glitch` firmware but uses the password-check function. The code compares a 5-character input to the stored password `"touch"` and sets `passok = 0` if any character mismatches. The result is returned via the SimpleSerial command `'p'`.

9.3 Attack Methodology

1. **Baseline**: Send wrong password → expect 0; send correct password → expect 1.
2. **Glitch configuration**: Use `scope.cglitch_setup()` and configure width, offset, and ext_offset. Use `GlitchController` to sweep the parameter space.
3. **Automated sweep**: For each parameter set, send a wrong password (e.g., all zeros), insert the glitch, and read the response.
4. **Outcome classification**:
   - **Success**: Valid response with "pass" (1) despite wrong password.

- **Reset**: Timeout or invalid response (target crashed).
- **Normal**: Valid response with "fail" (0).

5. **Visualization**: Plot success and reset points in (offset, ext_offset) space to identify effective fault regions.

9.4 Results Obtained

The sweep produced **successful bypass** outcomes: the target returned "pass" for an incorrect password when glitched at certain parameter combinations. Successes were clustered in specific regions of the (offset, ext_offset) space. Many attempts resulted in resets (target crash).

9.5 Analysis and Interpretation of the Results

Bypassing a password check is more security-relevant than corrupting a sum. The addition of ext_offset scanning increases the chance of landing the glitch at the critical comparison or branch. Separating useful faults (bypass with valid response) from destructive faults (crash) is essential for practical fault injection.

9.6 Challenges or Limitations

- The number of successful glitch points is smaller than in the summation lab; ext_offset scanning is important.
- Repeated resets and error handling are required for reliable automation.

9.7 Key Takeaways

- Clock glitching can bypass authentication without knowing the password.
- GlitchController and parameter sweeps enable systematic fault discovery.
- Fault injection poses a serious threat to embedded security; countermeasures (redundancy, fault detection) are necessary.

9.8 Conclusion

This lab applied clock glitching to bypass a firmware password check, demonstrating that authentication logic can be defeated by transient timing violations. Systematic parameter exploration using width, offset, and ext_offset produced successful bypass outcomes clustered in specific regions of the search space, while many attempts caused resets, emphasizing both the feasibility of bypass attacks and the need for robust defensive fault countermeasures.

10 Countermeasures

The labs in this report demonstrate that **side-channel leakage** and **fault injection** are realistic threats to embedded systems. Defending against them requires countermeasures at the **implementation and system** levels, not only the correct choice of cryptographic algorithms.

10.1 Side-channel countermeasures (SCA)

Attacks such as SPA, DPA, and CPA succeed because power (and related observables) correlates with **intermediate values** and **control flow**. Mitigations aim to break or reduce that correlation:

- **Masking**: Randomize intermediate values (e.g., secret sharing or Boolean/arithmetic masking) so that each trace carries only a share of the secret; a single measurement no longer reveals the unmasked value. Higher-order attacks exist and require higher-order masking or careful compositional proofs.
- **Hiding/noise injection**: Add amplitude or temporal noise (random delays, dummy operations) to increase the number of traces an attacker needs. This is often used as a complement to masking, not as a sole defense.
- **Constant-time and data-independent execution**: Avoid secret-dependent branches, table lookups indexed by secrets, and early exits (e.g., in password or MAC comparison). Use fixed-time comparison and uniform memory access patterns where the secret must influence behavior.
- **Leakage reduction at the circuit and layout level**: Lower supply current signature, balanced logic styles, and careful placement can reduce exploitable signals, but require hardware expertise and validation (e.g., TVLA / leakage assessment).
- **Protocol and operational measures**: Limit the number of queries, rate-limit sensitive operations, use fresh nonces/session keys, and combine with secure elements that offer evaluated implementations.

10.2 Fault-injection countermeasures

Clock glitching and related faults succeed when **timing violations** or **supply/clock manipulation** causes skipped instructions, corrupted state, or wrong branches. Defenses focus on **detection**, **integrity**, and **redundancy**:

- **Redundant checks and dual execution**: Perform critical checks twice (or compare against an independent path) so a single fault cannot skip both; use diversified code or temporal separation to reduce the chance one glitch hits all copies.
- **Integrity and control-flow protection**: Secure boot, runtime integrity monitoring (CRC/hash of code), and control-flow integrity (CFI) can detect unexpected jumps or corrupted code, though each must be designed to resist faulted verification itself.
- **Fault detection and monitoring**: Voltage and clock monitors, watchdogs, and anomaly detection can reset or lock the device when glitch-like conditions are detected.
- **Hardware hardening**: Glitch filters on clock and reset lines, internal oscillators, and tamper meshes increase attack cost. Certified secure elements often combine physical and logical protections.
- **Safe failure modes**: On detected fault, fail closed (no access, wipe sensitive state) rather than continuing in an undefined state.

10.3 Design principles for hardware security

- **Assume a physical attacker** with measurement and glitch capability when threat models require it (e.g., smartcards, HSMs, automotive keys).
- **Evaluate implementations**, not only algorithms: use formal leakage evaluation where possible and empirical tests (e.g., correlation/TVLA-style methodologies) during development.
- **Layer defenses**: No single countermeasure is perfect; combining masking, constant-time design, redundancy, and monitoring aligns with defense-in-depth for hardware security.

11 Conclusion

This report documented eight hardware security laboratories covering both **side-channel analysis (SCA)** and **fault injection**. On the SCA track, the labs demonstrated an escalation from intuitive, visual leakage exploitation (password recovery in Lab 2.1B) to increasingly powerful statistical attacks on AES: identifying data-dependent leakage (Lab 3.1), recovering information from a minimal leakage assumption (Lab 3.2), performing classic DPA (Lab 3.3), validating Hamming-weight leakage behavior (Lab 4.1), and finally recovering the AES key efficiently via CPA (Lab 4.2). Collectively, these steps show how the attacker's success improves when the leakage model becomes better aligned with the implementation and when the distinguisher extracts more information from each trace.

On the fault-injection track, the labs introduced **clock glitching** as a technique for pushing a microcontroller briefly outside its timing assumptions. The first fault lab established the underlying mechanics of configuring glitch parameters and classifying outcomes. The second fault lab applied those mechanics to a password check, demonstrating that faults can directly change program outcomes and bypass authentication logic without altering the cryptographic algorithm itself.

Across the entire set of labs, several consistent themes emerged:

- **Physical observables are attack surfaces.** Power consumption and timing behavior encode information about internal state and control flow. Even small, repeatable differences become exploitable when aggregated across many measurements.
- **Model-driven analysis is essential.** Choosing and evaluating a leakage hypothesis (bit-level separation, DPA partitioning, Hamming-weight modeling, or predicted leakage for CPA) turns raw measurements into actionable key-recovery decisions.
- **Noise and artifacts must be managed.** Results can be affected by trace alignment, measurement polarity, and statistical artifacts such as "ghost peaks." Robust preprocessing, trace count, and careful interpretation are required for reliable conclusions.
- **Fault injection is inherently unstable and operationally complex.** Effective experiments rely on systematic parameter exploration, dependable resets, and error-tolerant communication to separate useful fault outcomes from crashes/timeouts.

The experiments underscore why embedded security must be engineered with implementation details in mind. Countermeasures such as **side-channel hardening** (masking, hiding, constant-time design) and **fault resilience** (fault detection, redundancy, integrity checking, and robust error handling) address the practical ways attacks succeed in the physical world. In this context, the ChipWhisperer platform served as a practical environment to connect theory with measurement-based evidence, reinforcing that the security boundary includes the device's real execution and physical behavior, not just the algorithmic specification.

Taken together, these labs reinforce a defense-in-depth perspective: security evaluations must account for both leakage and fault sensitivity. When physical implementation properties are ignored, systems that are theoretically secure in software can still fail under real-world measurement and fault conditions. These results provide hands-on evidence that ensuring security requires not only correct cryptographic algorithms, but also careful implementation-level techniques that reduce or decorrelate leakage and that detect, contain, or neutralize induced faults so that security-critical decisions remain correct under attack.